

Mecanum-wheeled Autonomous Tracking Robot

项目组长 姚康晨

项目成员 金振宇 王明睿 赵禹赫 罗子轩

汇报日期 2026.5.23

Trace Rover Team

Contents

01

Project Overview

Background & Objectives

02

Structure

Chassis & Scanning Platform

03

Control

Kinematics, PID & Navigation

04

Electronic

PCB & Hardware Platform

05

Software

GUI & Visual Perception

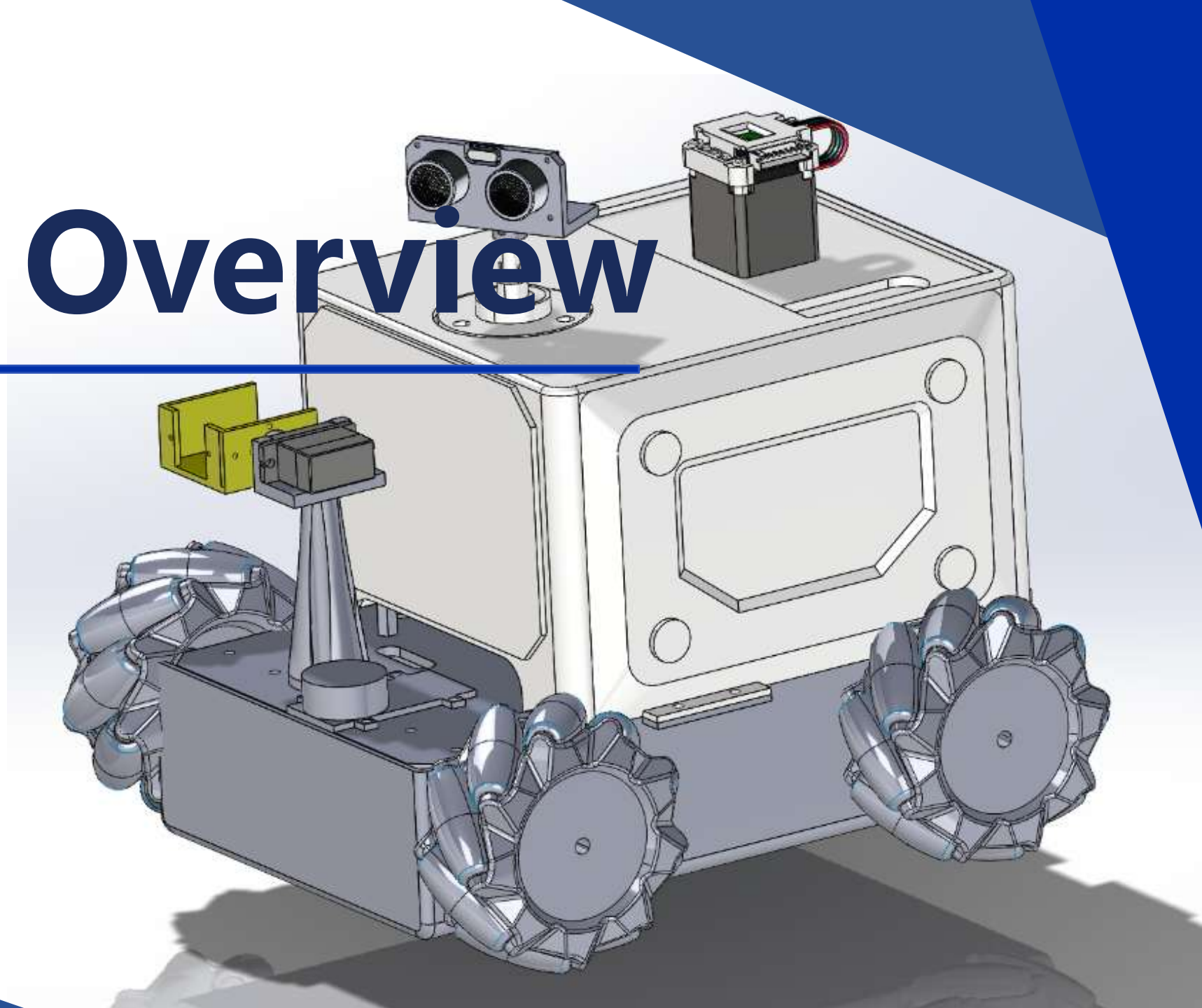
06

Budget

Part One

Project Overview

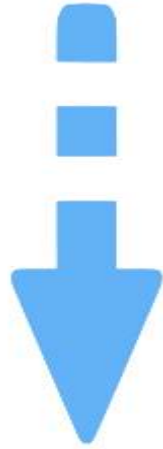
- Background & Motivation
- Goals & Key Features



01 Background & Motivation

In factory environments, safety inspection is essential but repetitive and time-consuming.

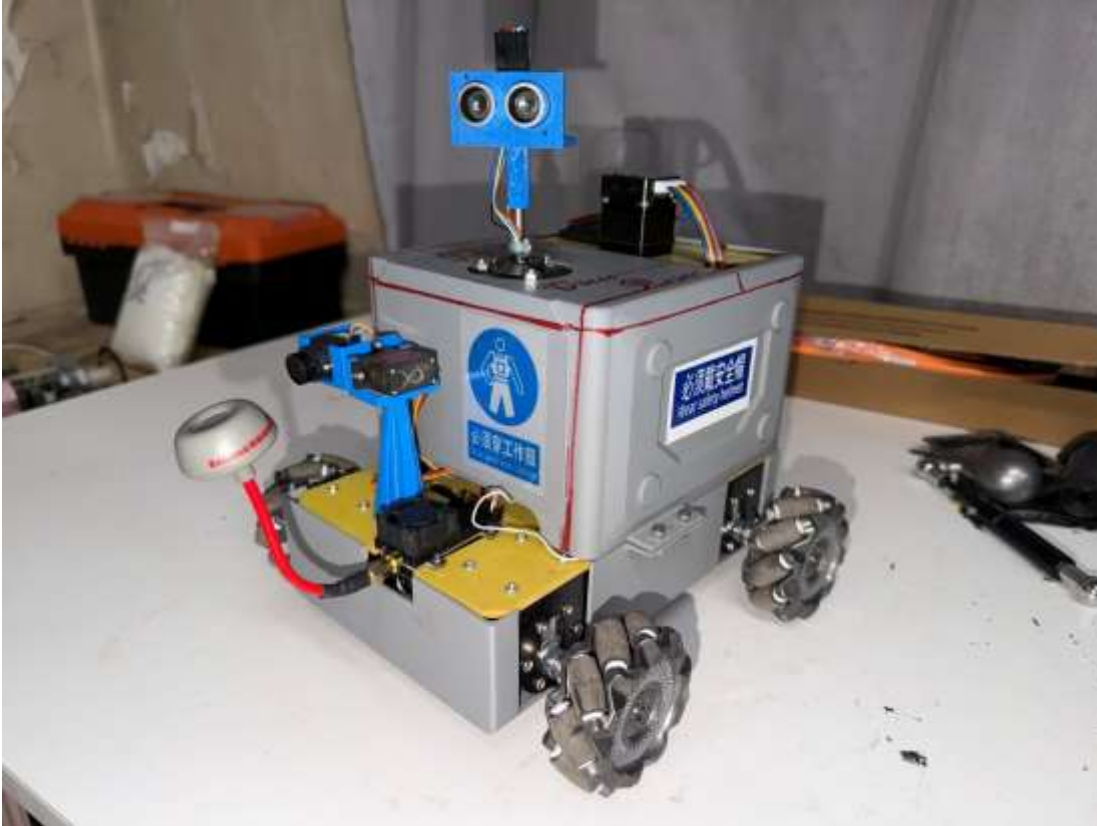
Manual inspection may suffer from limited efficiency, blind spots and delayed response.



Growing demand for autonomous robots to monitor industrial environments



01 Goals & Key Features



Visual Recognition

Safety Helmet & PPE Detection

- YOLO model for real-time detection
- Identifies helmet & uniform compliance

Obstacle Avoidance

360° Omnidirectional Scanning

- Stepper motor + timing belt platform
- Eliminates blind spots in all directions

Autonomous Tracking

Mecanum-Wheel Omni-Drive

- Holonomic motion in any direction
- Cascade PID closed-loop control

Part Two

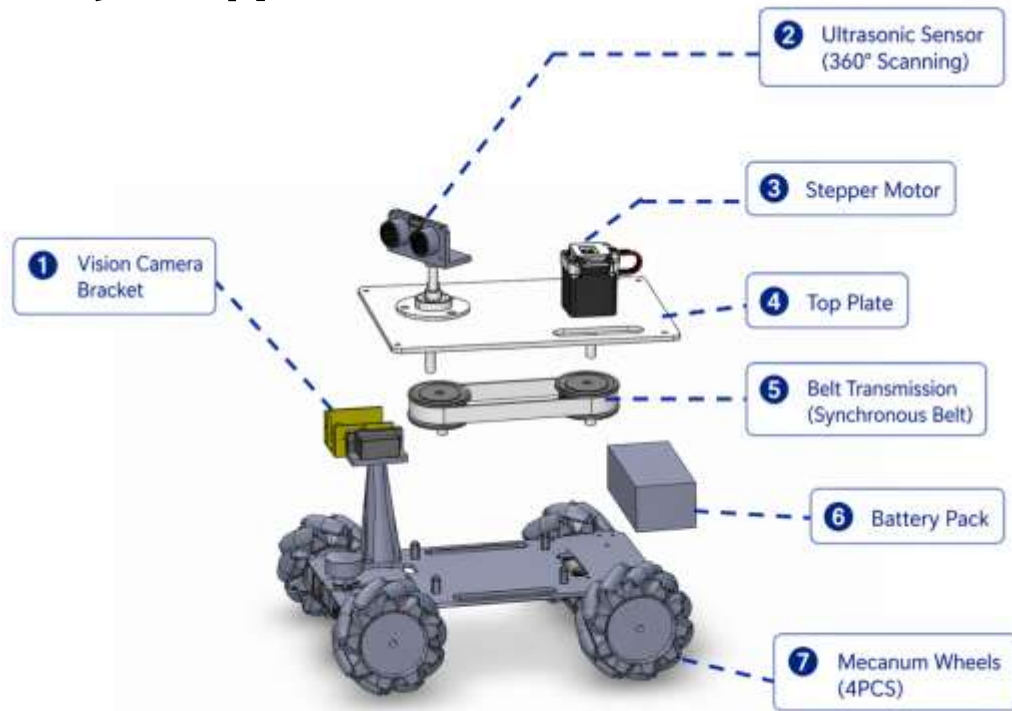
Structure

罗子轩
金振宇

- Chassis Architecture Specifications
- 360° Scanning Platform
- Structural Integration & Key Mechanical Components

02 Chassis Architecture Specifications

罗子轩



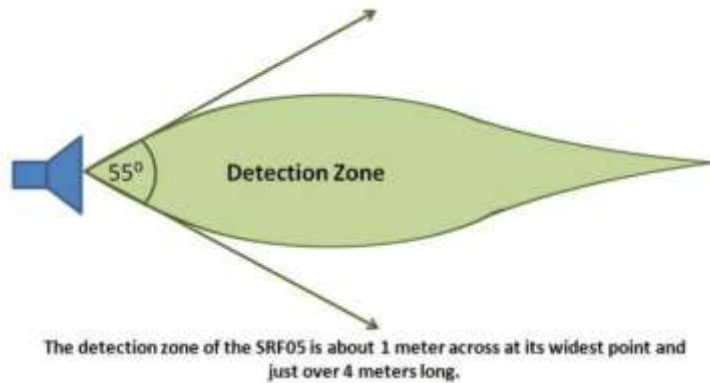
Double-Layer 3D Structure — Heavy components on bottom layer to lower center of gravity; control board and sensors on top

Vision Mount — Cantilever bracket at front for camera; ensures unobstructed view of workers' PPE status

Load Distribution — Balanced load across 4 Mecanum wheels to prevent slippage

02 360° Scanning Platform

The Problem



A single fixed ultrasonic sensor covers only ~55° forward, leaving over 305° as blind spots.

SOLUTION



Transmission:
Stepper motor +
timing
belt/pulley —
motor mounted
side to minimise
platform height.

Engineering Advantages



360° Coverage

The ultrasonic sensor scans surrounding obstacles instead of only detecting the front.



Compact Layout

The side-mounted motor and belt transmission reduce the overall height of the platform.

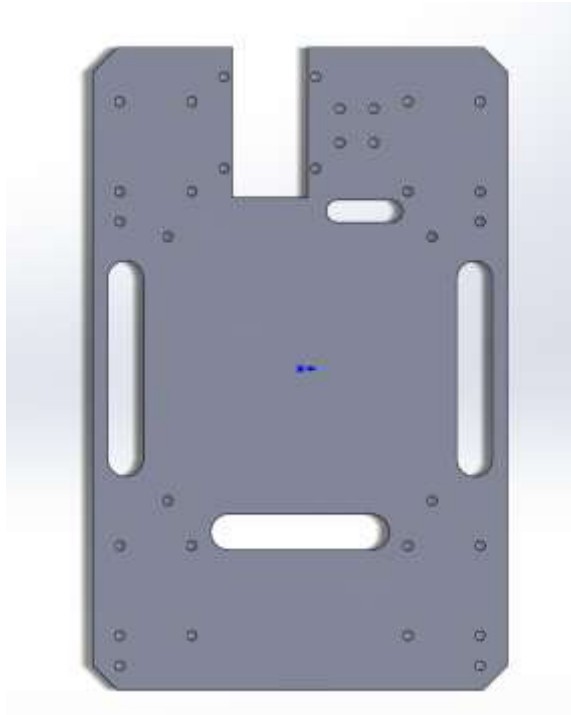


Stable Scanning

The timing belt reduces vibration and improves scanning smoothness.

03 Structural Integration & Key Mechanical Components

My Work 金振宇



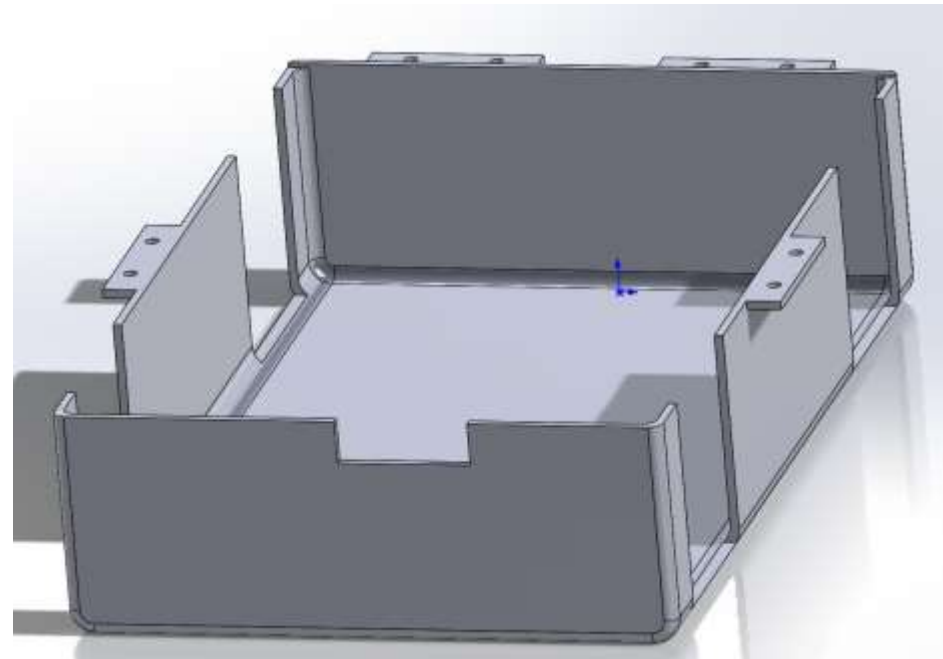
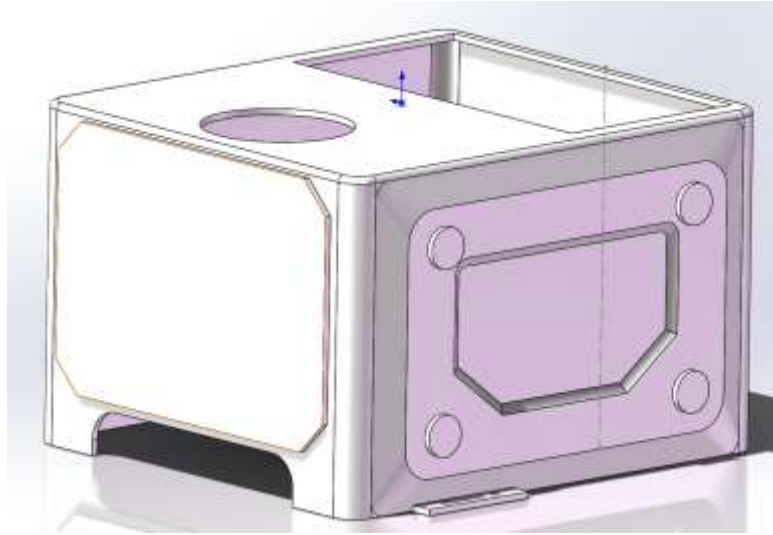
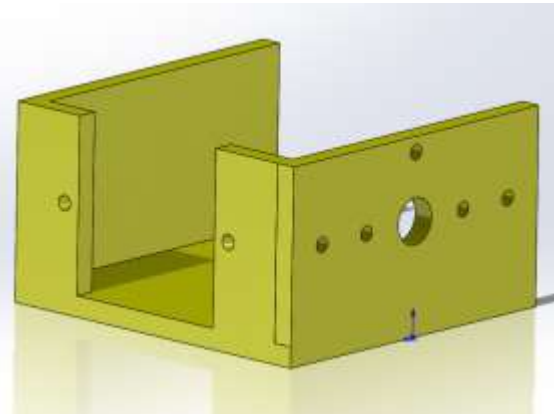
Part drawing:

1. Chassis
2. Camera stand
3. Camera base
4. Chassis housing
5. Shell



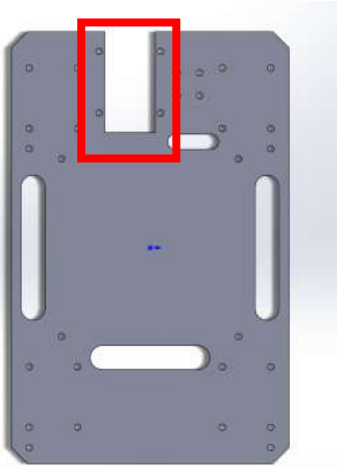
Assembly:

1. SW's 3D Assembly
2. Model assembly



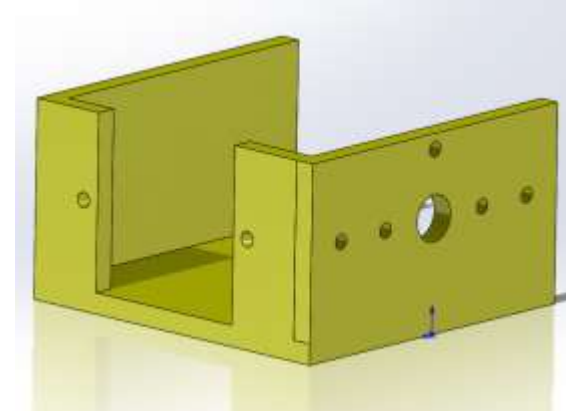
02 Structural Integration & Key Mechanical Components

The problem I encountered



Chassis: There are issues of part interference and size discrepancies in the red box area.

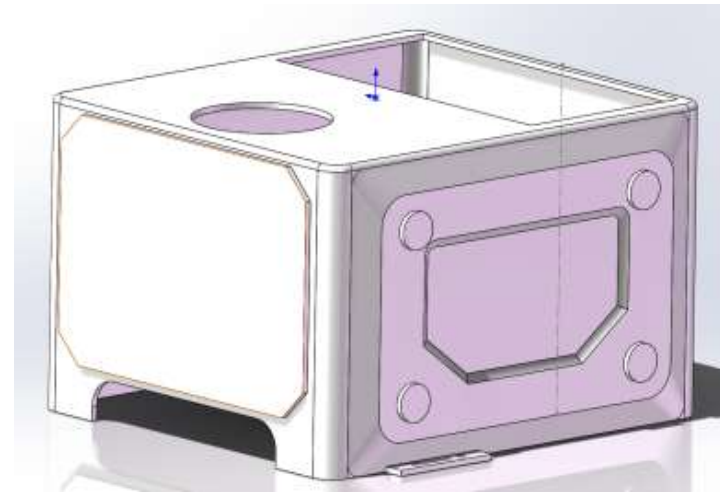
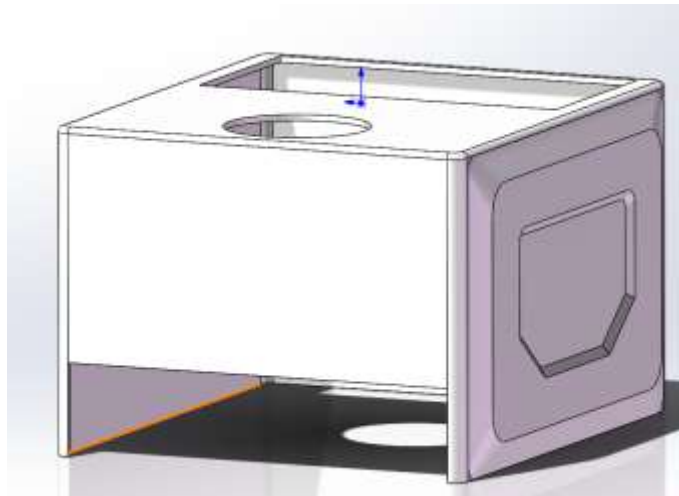
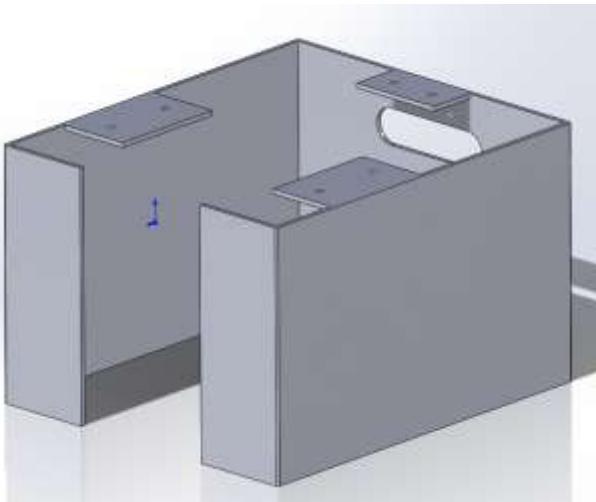
Solution: Manual polishing



Camera base: The issue with the size of the camera base is that the rear part extends too long.

Solution: Manual polishing

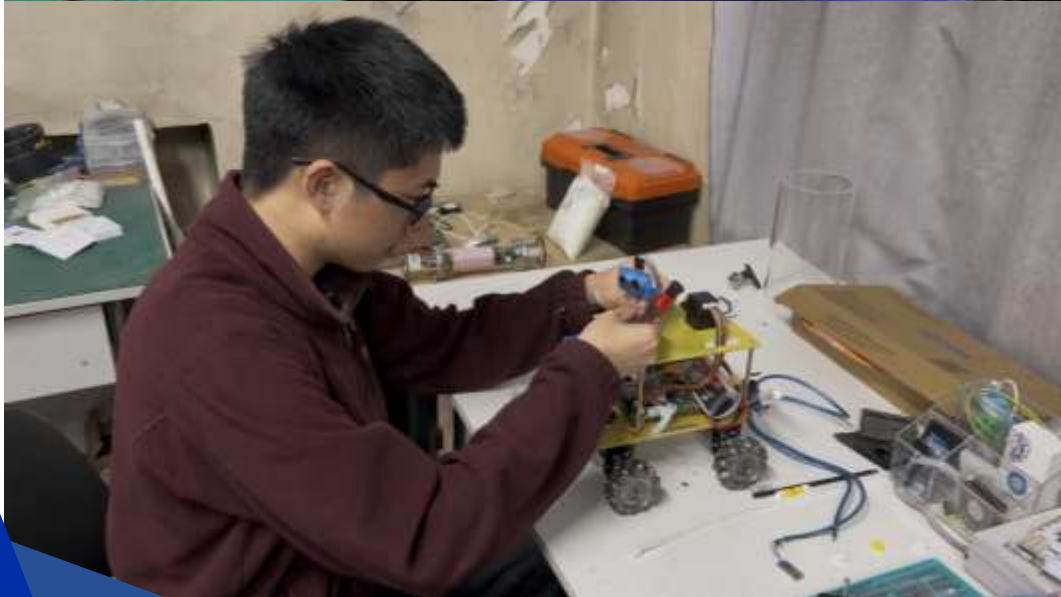
Shell iteration



The problem I encountered

1. The shell has undergone three major version iterations.
2. The final version of the casing was adjusted twice regarding the size issue.

Assembly process



Part Four

Electronic

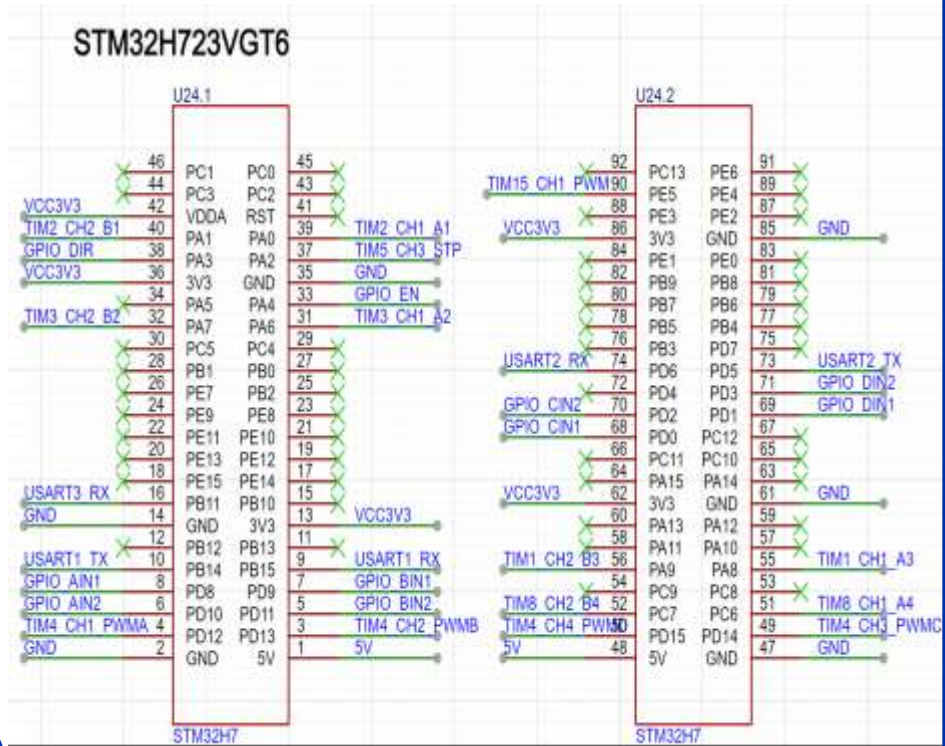
赵禹赫

- Schematic Low-Level Optimization Details
- High-Reliability PCB Layout Technology
- Deployment with STM32CubeMX
- Signal Integrity & Anti-Interference
- PCB Design Problems & Optimization

04 Schematic Low-Level Optimization Details

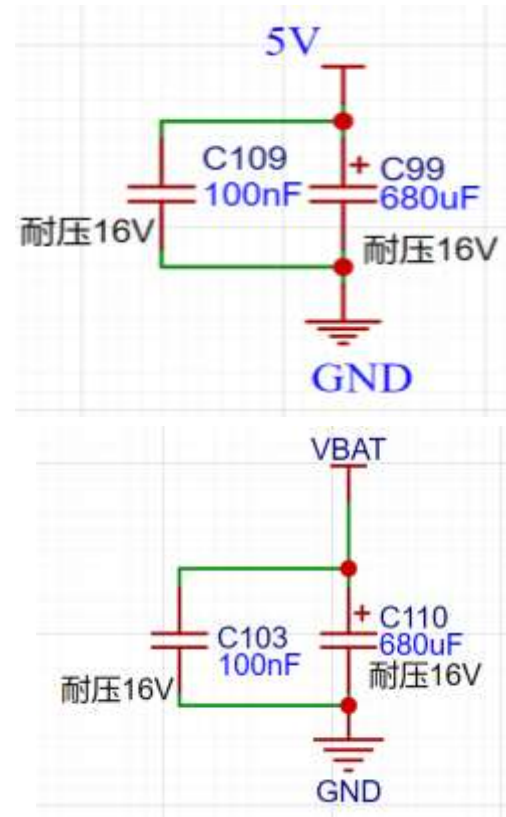
Computing Core:

Adopted 550MHz STM32H723VGT6 MCU to support vector control response.



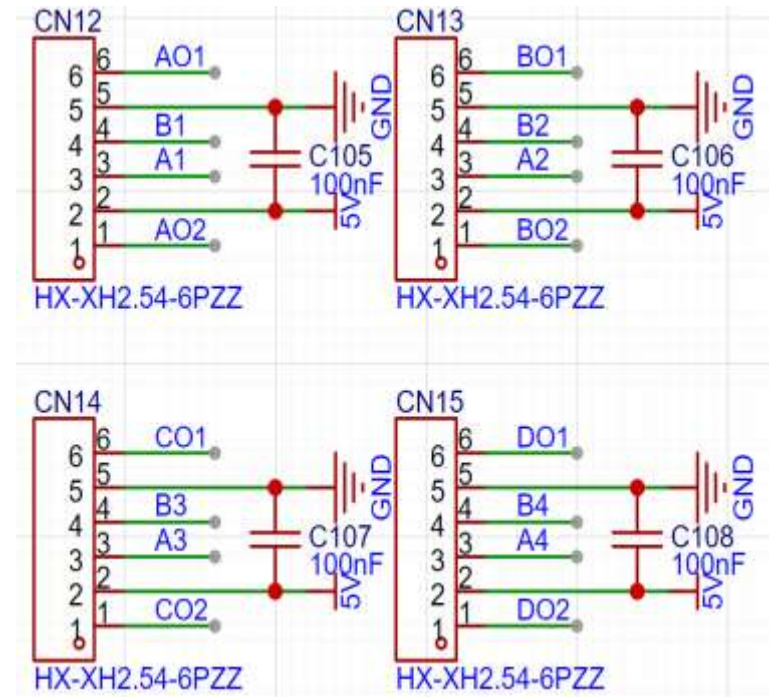
Power Integrity:

Adopted MP1584 DC-DC converter, equipped with capacitors to suppress motor ripples.



Encoder Protection:

Applied 100nF filter capacitors to shield speed feedback signals from motor interference.



04 High-Reliability PCB Layout Technology

Stitching Via Array:

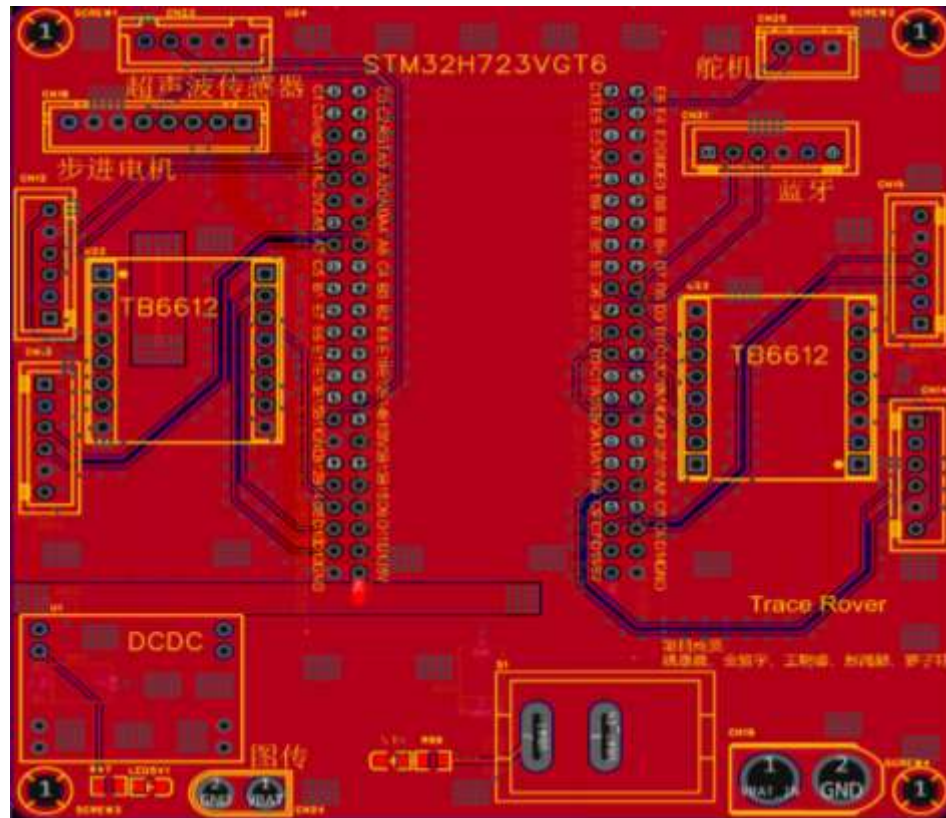
Arranged stitching vias to lower ground impedance, enhance heat dissipation and restrain noise.

Signal Guarding:

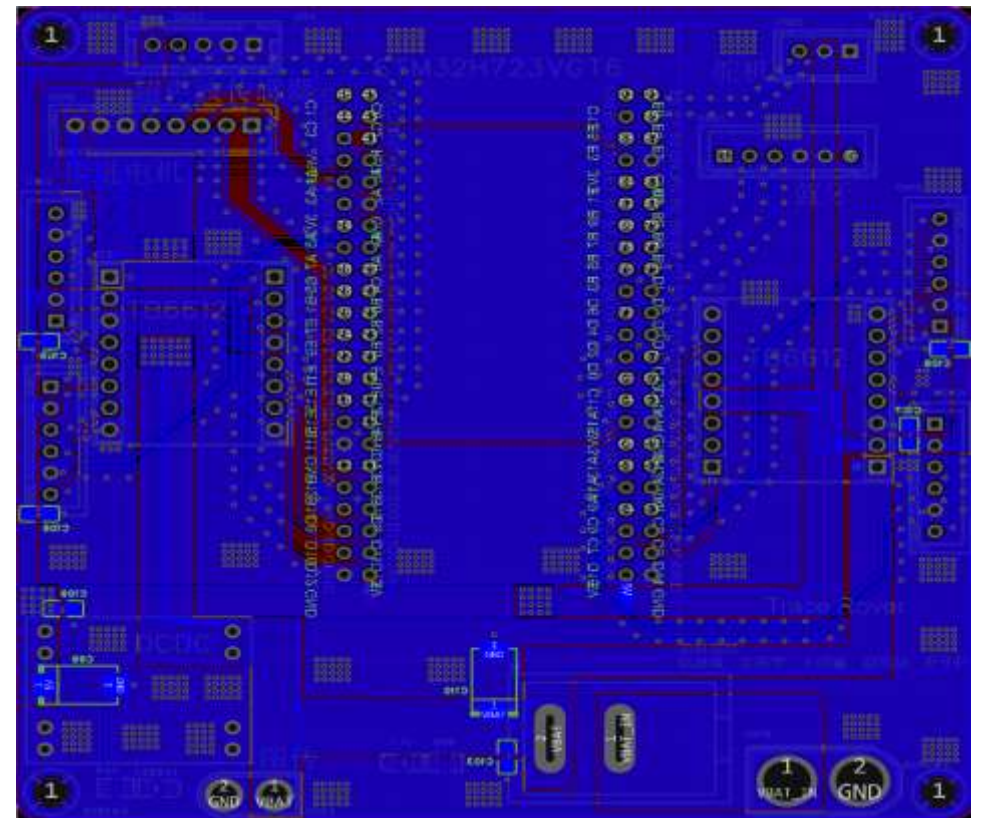
Applied via stitching beside key signals to isolate high-current interference.

Physical Partitioning:

Symmetrical layout separates control and drive zones for strong-weak electricity isolation and EMI reduction.



Top Signal Layer

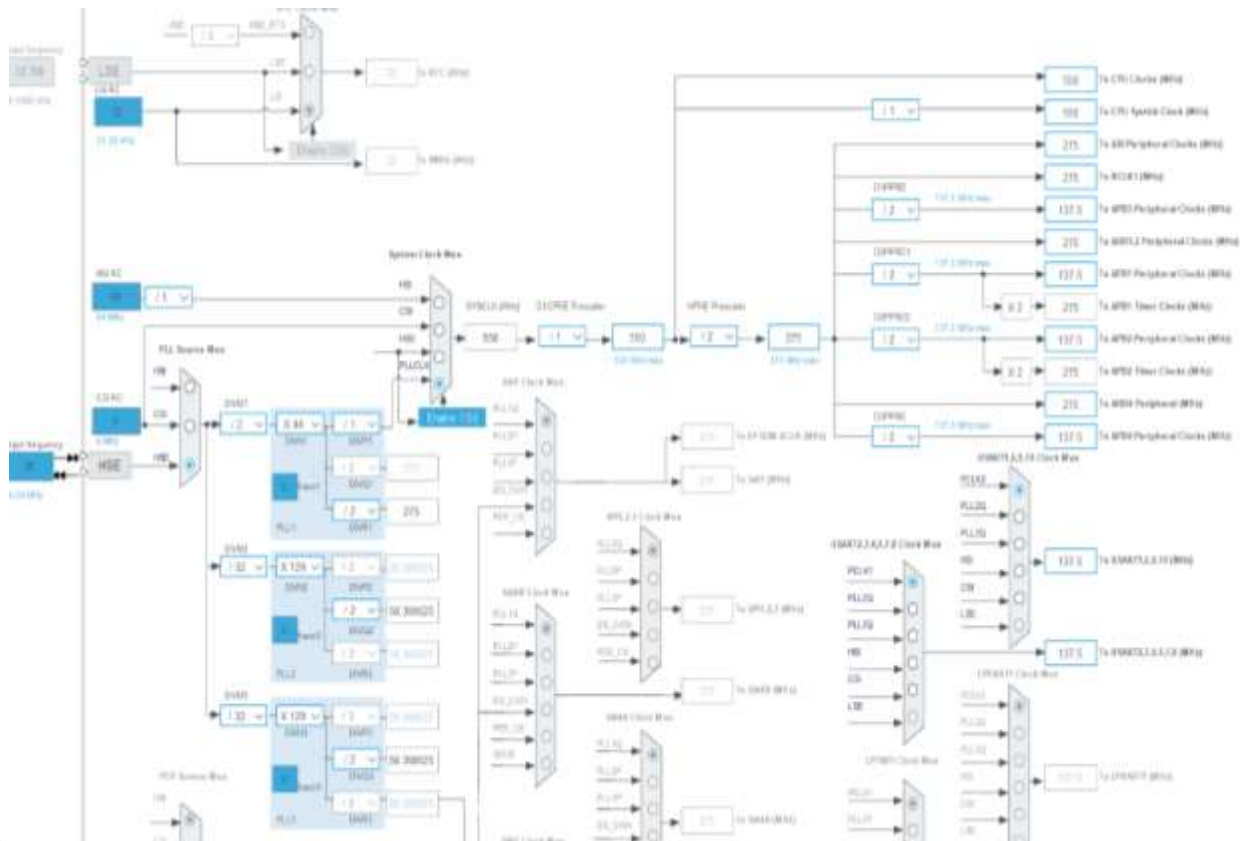


Bottom Power Layer

04 Deployment with STM32CubeMX

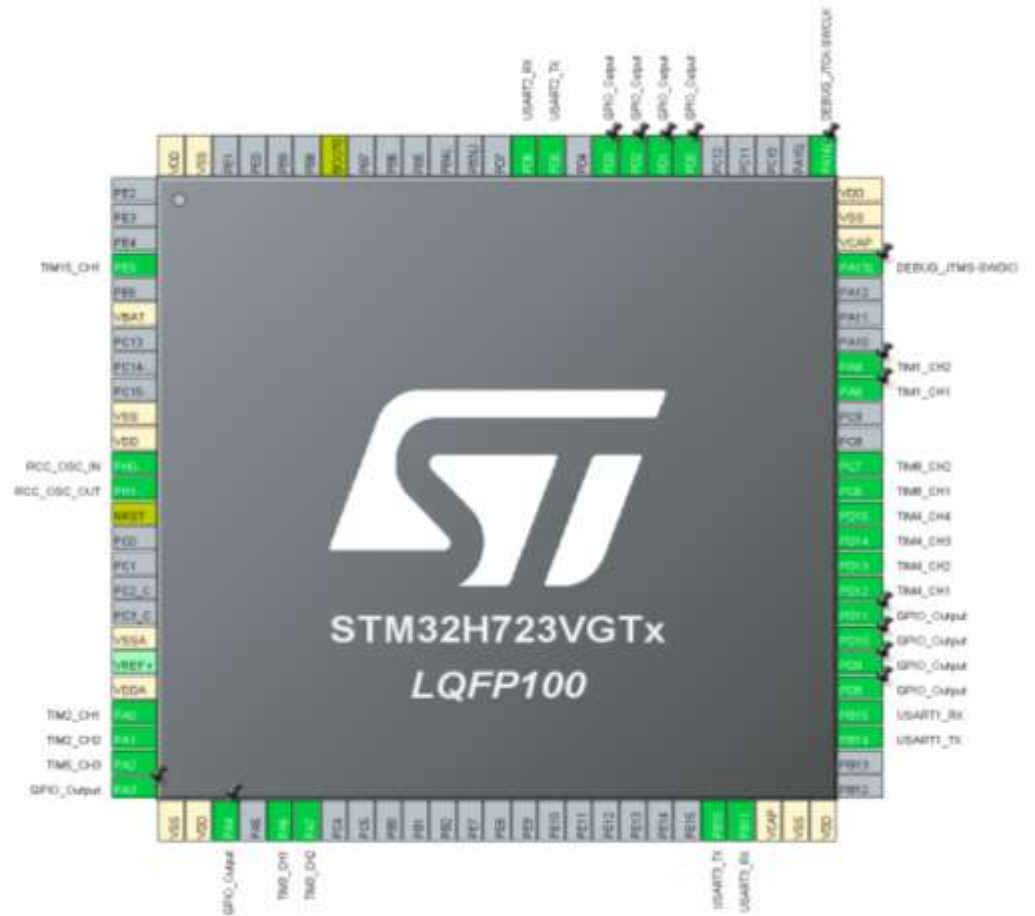
550MHz Core Performance:

550MHz clock tree full allocation, full frequency operation.



Unified Peripheral Scheduling:

STM32CubeMX allocation of pin functions to generate the source code of Hal Library.



04 Signal Integrity & Anti-Interference

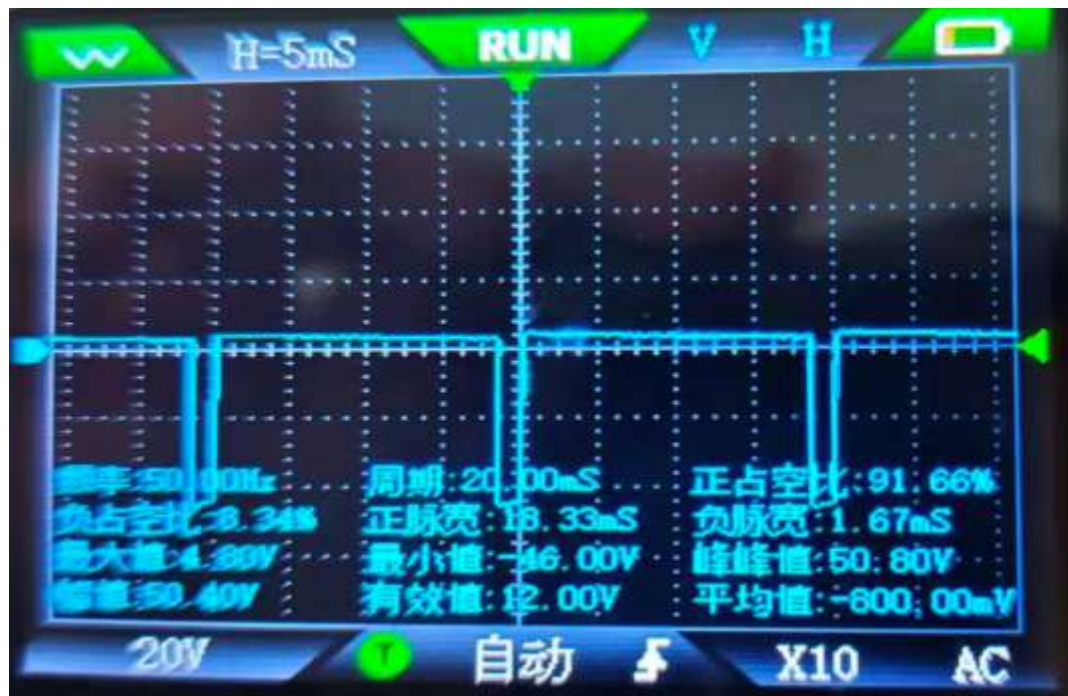
PWM Drive Waveform

Measured Data:

Outputs 50.00Hz **PWM** motion control waveforms.

Significance:

Fine impedance matching.



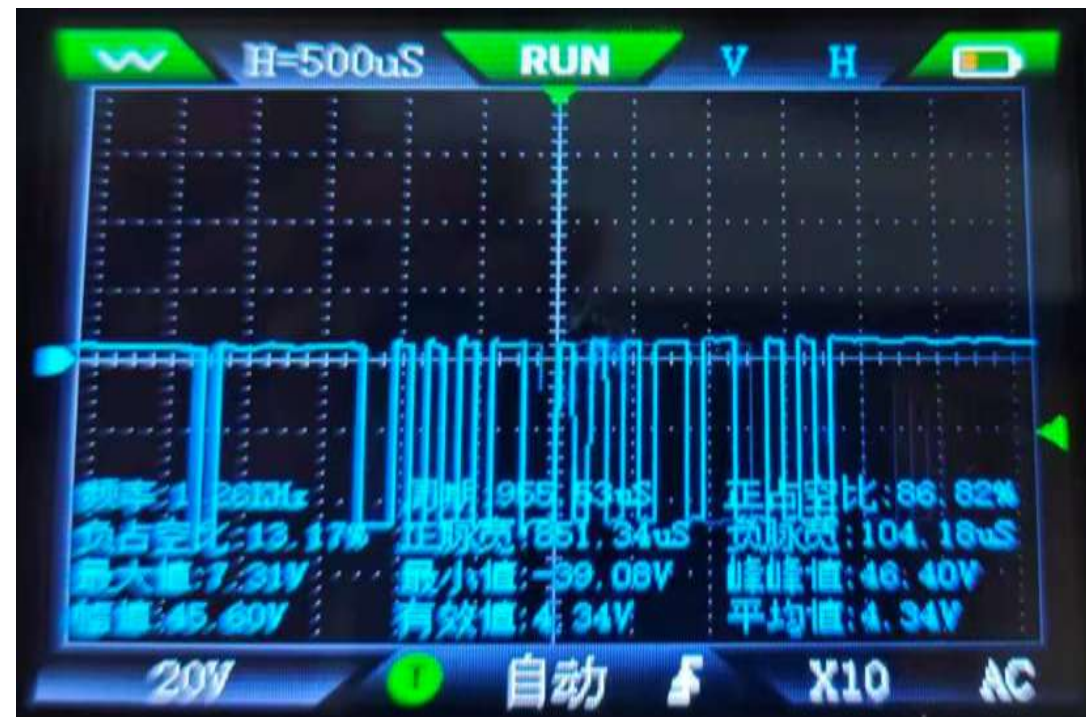
Bluetooth UART Waveform

Measured Data:

Captured low-level digital logic pulses from **UART**.

Significance:

Signals remain distortion-free.



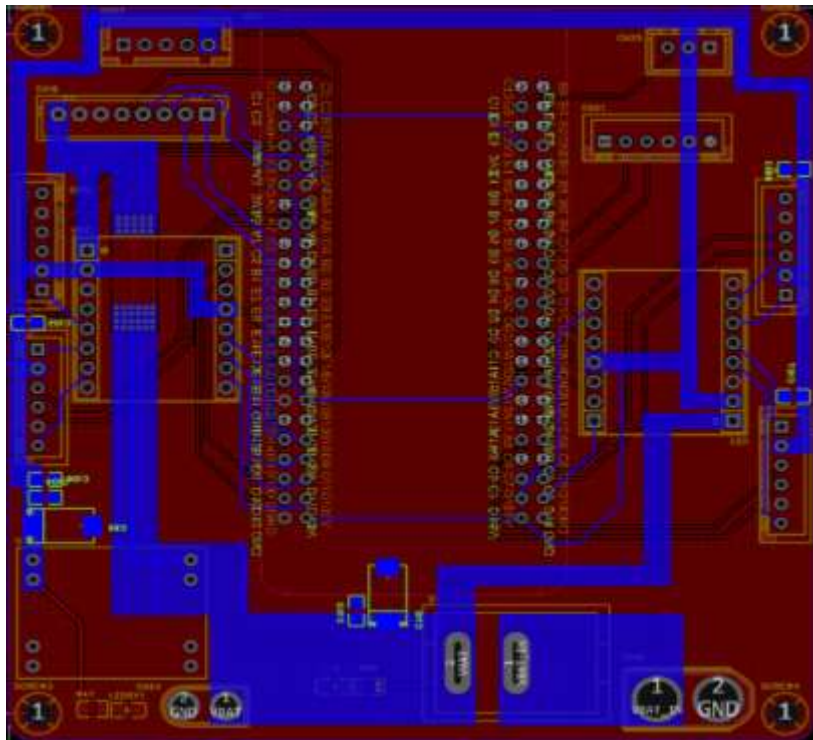
04 PCB Design Problems & Optimization

Problems

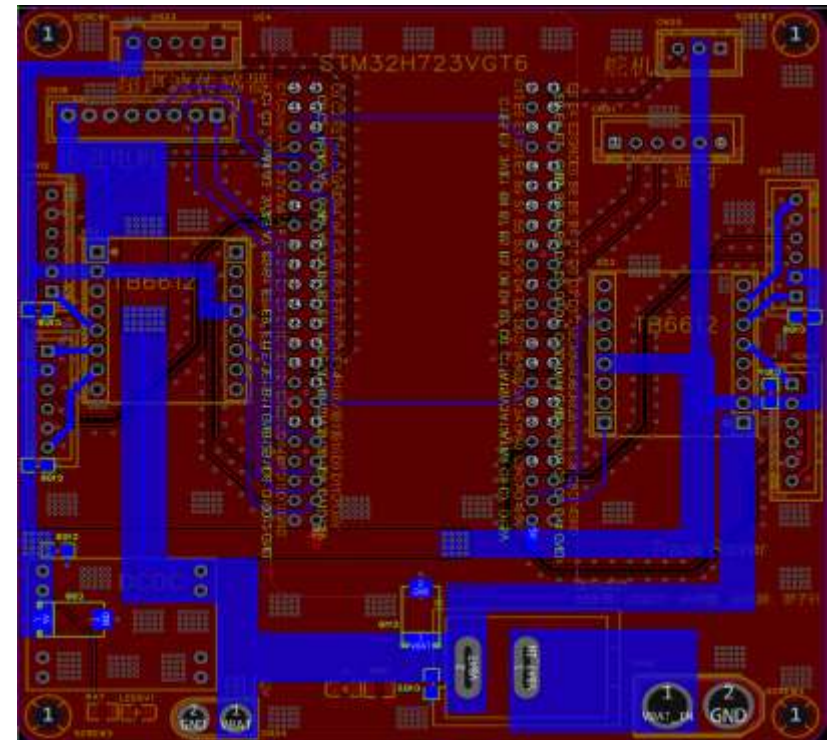
- Power trace forms closed loop leading to high EMI
- Serious heating of circuit board
- Signal lines are susceptible to interference

Optimization Measures

- Break all power loops by re-routing traces to follow ground plane
- Add stitching vias along board edges
- Vias are added on both sides of the signal line



Upgrade



Part Three

Control

姚康晨

- MCU System Architecture
- Task Scheduling: Time-Slice Round-Robin
- Motor Closed-Loop Control: Cascade PID
- Navigation: Encoder Odometry + Aruco Correction

03 MCU System Architecture

User APP

Layer 3 — Application

Task scheduling · Motion logic · Communication protocol

HAL

Layer 2 — Hardware Abstraction

Sensor drivers · Encoder feedback · PWM generation

BSP / Peripherals

Layer 1 — Driver

STM32 timers · UART · GPIO low-level control

High cohesion, low coupling — all modules encapsulated via object-oriented design

```
▼ Trace_Rover
  ▼ Core
    ▼ Src
      C gpio.c
      C main.c
      C stm32h7xx_hal_msp.c
      C stm32h7xx_it.c
      C system_stm32h7xx.c
      C tim.c
      C usart.c
    > Drivers
    > MDK-ARM
  ▼ User_APP
    > Inc
    ▼ Src
      C comm.c
      C tasks.c
  ▼ User_HAL
    > Inc
    ▼ Src
      C actuator.c
      C coordinate2d.c
      C fifo.c
      C filter.c
      C fsm.c
      C mecanum.c
      C pid.c
      C time_slice.c
      C ultrasonic.c
```

1

1000 Hz System Tick

STM32 timer interrupt fires every 5 ms, used as the global timestamp base for all task slots.

2

Non-Blocking Scheduling

Each task runs within its allocated time slice without blocking others — ensures real-time responsiveness.

3

Modular Task Design

Motion control, sensor reading, and communication each run as independent modules — easy to debug and extend.

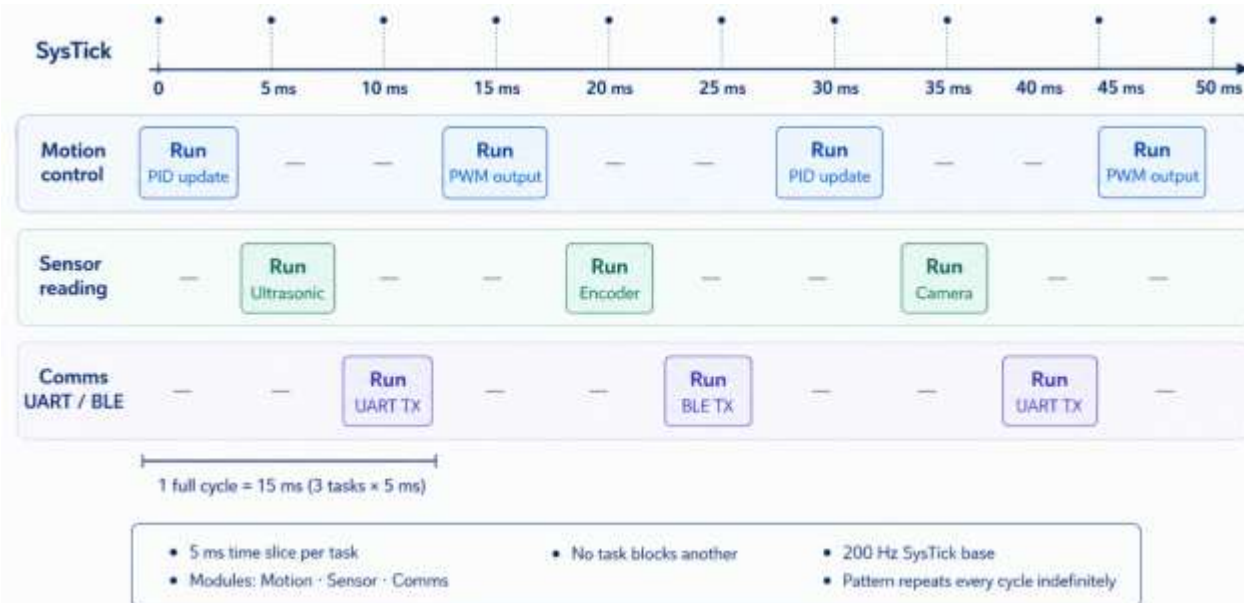
Single-loop PID balances dynamic response and positioning accuracy

```

* @brief 时间片任务初始化
*
* @param ts 时间片结构体
* @param interval 时间间隔, 单位: ms
* @param callback 时间片任务回调函数
*/
// 解算代码 [注册代码] X
void time_slice_init(time_slice *ts, uint16_t interval, void (*callback)(void))
{
    ts->last_time = HAL_GetTick();
    ts->interval = interval;
    ts->callback = callback;
}

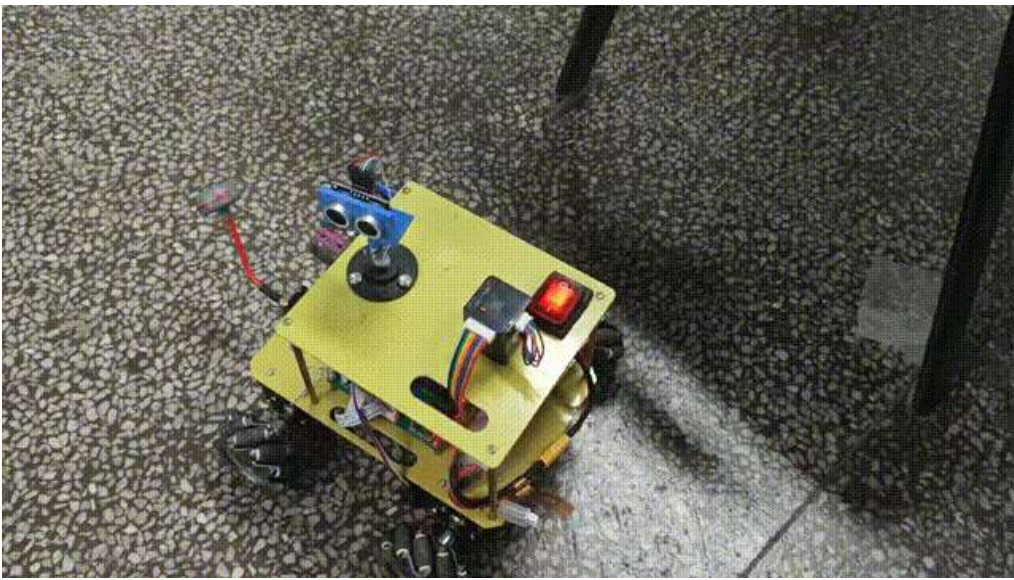
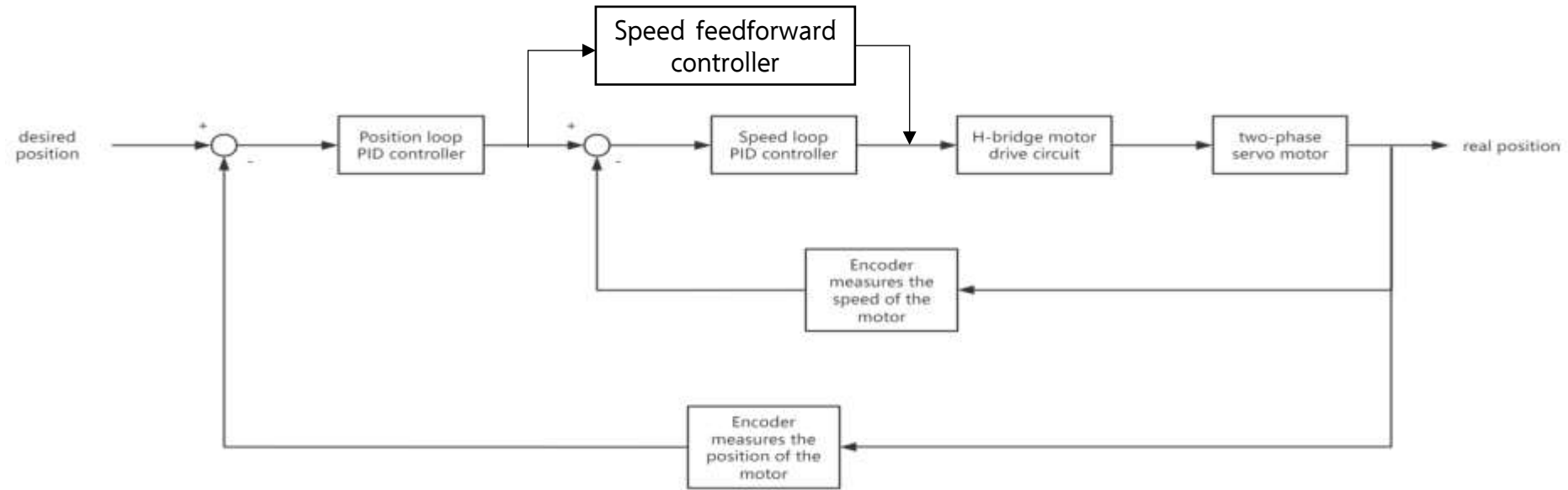
/**
* @brief 时间片任务运行
*
* @param ts 时间片结构体指针
*/
// 解算代码 [注册代码] X
void time_slice_run(time_slice *ts)
{
    if (HAL_GetTick() - ts->last_time >= ts->interval)
    {
        ts->last_time = HAL_GetTick();
        ts->callback();
    }
}

```



Single-loop PID balances dynamic response and positioning accuracy

03 Motor Closed-Loop Control: Cascade PID



problem

1. The system response speed is too slow.
2. The motor is experiencing severe vibration.
3. The braking speed is too slow.

solution

1. Apply the feedforward controller.
2. Apply the output smoothing algorithm.
3. Apply the integral anti-saturation algorithm

Inertial navigation system based on encoder

To obtain the position and speed information of the car as feedback values for closed-loop control, we used the encoder on the motor to implement an incremental odometry inertial navigation system.

This system operates stably at a frequency of 200Hz, but the error accumulation caused by slippage and noise is unacceptable.

Therefore, our vision group and software group implemented its aruco code navigation system, solving the drift problem of inertial navigation.

```
static float last_wheels[4] = {0.0f, 0.0f, 0.0f, 0.0f};

// 保存当前轮子位置用于增量计算
float current_wheels[4];
for(uint8_t i = 0; i < 4; i++)
{
    current_wheels[i] = motor[i].real_position;
}

// 更新 mecanum_pos (保持兼容性)
for(uint8_t i = 0; i < 4; i++)
{
    mecanum_pos.wheels[i] = current_wheels[i];
}
mecanum_forward_kinematics(&mecanum_pos, mecanum_pos.wheels);

// 计算轮子位置增量
float wheel_deltas[4];
for(uint8_t i = 0; i < 4; i++)
{
    wheel_deltas[i] = current_wheels[i] - last_wheels[i];
    last_wheels[i] = current_wheels[i];
}

// 使用增量计算底盘位置 (机器人坐标系)
mecanum temp_mecanum;
mecanum_forward_kinematics(&temp_mecanum, wheel_deltas);

// 将机器人坐标系的增量转换到世界坐标系
base_sys2D delta_base, delta_world;
base_sys_set_by_vector(&delta_base, temp_mecanum.chassis.x, temp_mecanum.chassis.y, temp_mecanum.chassis.r);
base2world(&rover_pos, &delta_base, &delta_world);

// 更新底盘位置 (世界坐标系)
rover_pos.vector.x = delta_world.vector.x;
rover_pos.vector.y = delta_world.vector.y;
rover_pos.vector.r = delta_world.vector.r;

// 角度校正
rover_pos.vector.r = angle_correct_rad(rover_pos.vector.r);

// 更新齐次变换矩阵
vector2matrix(&rover_pos);
```

Part Five

Software

王明睿

- Communication Module
- Upper Computer Software
- Navigation Visualization & Mission Planner
- Vision Process: YOLO Detection & Aruco Navigation

Stable Communication

As we adopted the Bluetooth module as the communication method, in order to achieve BLE protocol communication between the upper computer and the lower computer, we applied the bleak library and, based on the data frame format we defined, developed the communication module. This module serves as the foundation for the upper computer to control and monitor the AGV.



```
class trace_rovers:
    def notification_handler(self, sender, data):
        if not self._connected:
            return
        # 看原始字节是什么
        # print(f'DEBUG Received Raw: {data.hex(' ').upper()}')
        self._buffer.extend(data)
        self._parse_buffer()
        self._notification_received = True

    @trace_解释代码|注释代码|×
    def _reader_thread(self):
        while self._running:
            try:
                if not self._client.is_connected:
                    time.sleep(0.005)
            except Exception as e:
                print(f'BLE Error: {e}')
                time.sleep(1)
    @trace_解释代码|注释代码|×
    def _parse_buffer(self):
        """
        [解包逻辑]
        最小帧长 = Head(2) + Cmd(1) + Data[至少1] + Check(1) + Tail(2) = 7字节
        """
        while len(self._buffer) >= 7:
            # Step 1: 找帧头
```

寄存器地址/HEX	名称	结构体名称	数据类型	数据长度/位	内容结构组成	寄存器/位
0x4E	导航信息	navigation_info	u8	1	读写标志: 0x00, 写入; 0x01, 读出	25
			f32[3]	12	位置信息 x, y, z	
			f32[3]	12	速度信息 x, y, z (在写入时不生效)	
			f32	4	距离: distance	
			f32	4	角度: angle	
			f32	4	x坐标位置	
			f32	4	y坐标位置	
0x55	超声波信息	ultrasonic_info	u8	1	读写标志: 0x00, 写入; 0x01, 读出	16
			u8	1	控制寄存器 (0: 速度控制; 1: 位置控制)	
			f32	4	参数: err_d 误差 (在写入时不生效)	
			f32	4	参数: err_t 误差 (在写入时不生效)	
			f32	4	参数: Kp	
			f32	4	参数: Ki	
			f32	4	参数: Kd	
			f32	4	参数: i_limit 积分上限	
			f32	4	参数: d_limit 输出上限	
0x50	PID控制器信息	pid_info	u8	1	0: 0x0E, 速度闭环 (世界坐标系); 0x03, 位置闭环 (机器人坐标)	34
			f32[3]	12	控制量 (数据意义由工作模式决定; 前速指令/前速速度/前速位)	
0x40	运动控制指令	control_info	u8	1		13
0x53	舵机控制指令	servo_info	f32	4	舵机角度	4

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    static FIFO_RESULT ret;
    if (huart == &huart1)
    {
        ret = Fifo_RefreshExternal(&uart1_fifo);
        if (ret == FIFO_RECEIVED)
        {
            // 处理数据
            uart1_solve(uart1_fifo.fifo);
        }
        // 再次接收
        HAL_UART_Receive_IT(&huart1, (uint8_t *)&uart1_fifo.fifo[uart1_fifo.point]);
    }
    if (huart == &huart2)
    {
        ret = Fifo_RefreshExternal(&uart2_fifo);
        if (ret == FIFO_RECEIVED)
        {
            uart2_solve(uart2_fifo.fifo);
        }
        // 再次接收
        HAL_UART_Receive_IT(&huart2, (uint8_t *)&uart2_fifo.fifo[uart2_fifo.point]);
    }
    if (huart == &huart3)
    {
        if (uart3_buffer[uart3_ptr] == 0xFF)
```


05 Upper Computer Software

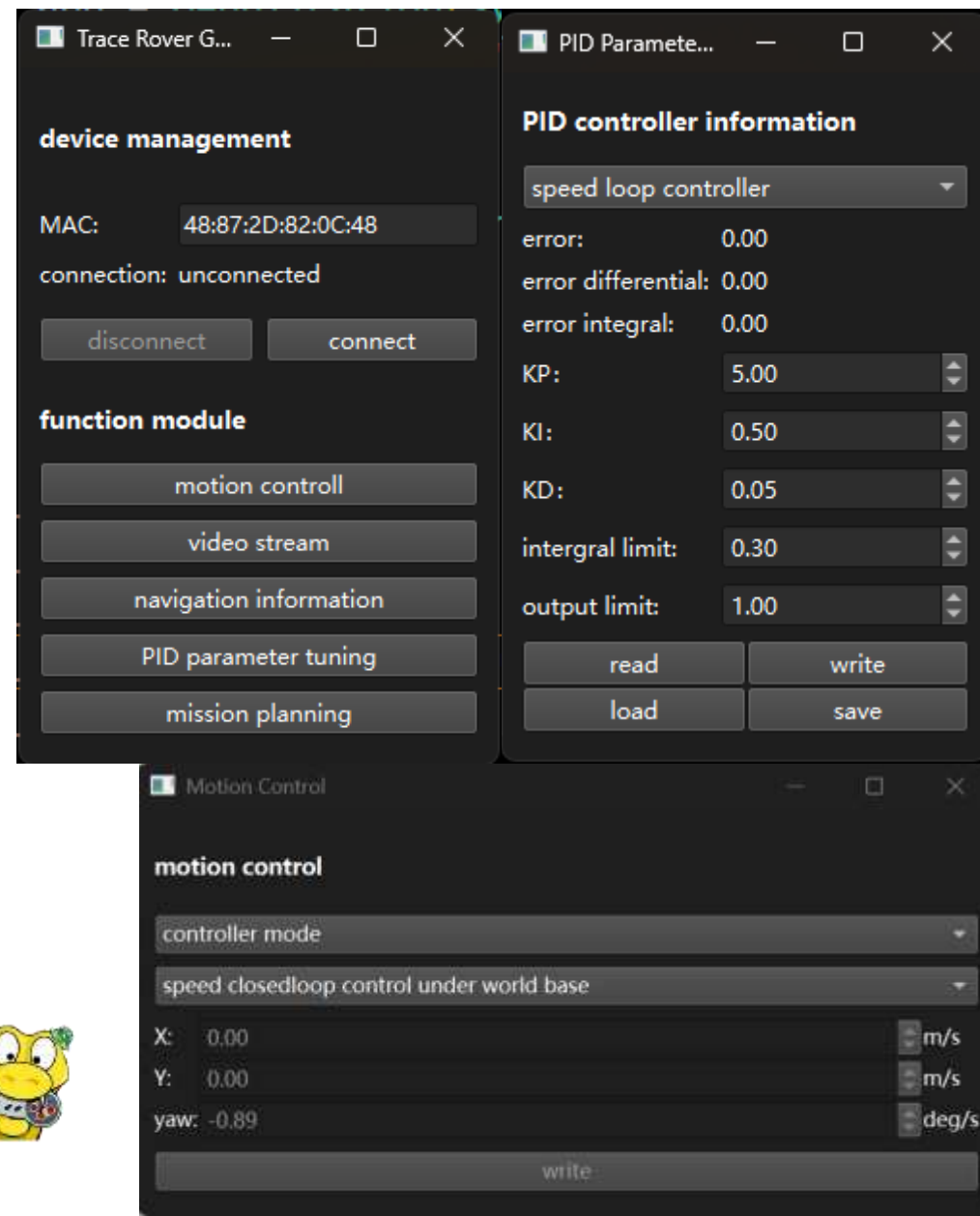
AGV upper computer software

We have developed a **cross-platform, visual,** and **user-friendly** upper computer control software using PySide6.

The four core modules are placed in five separate windows and are controlled by a main window.

In the **motion control** interface, users can manually input control values in the interface and send them. We also use the joystick module in Pygame to allow users to control the AGV with a gamepad.

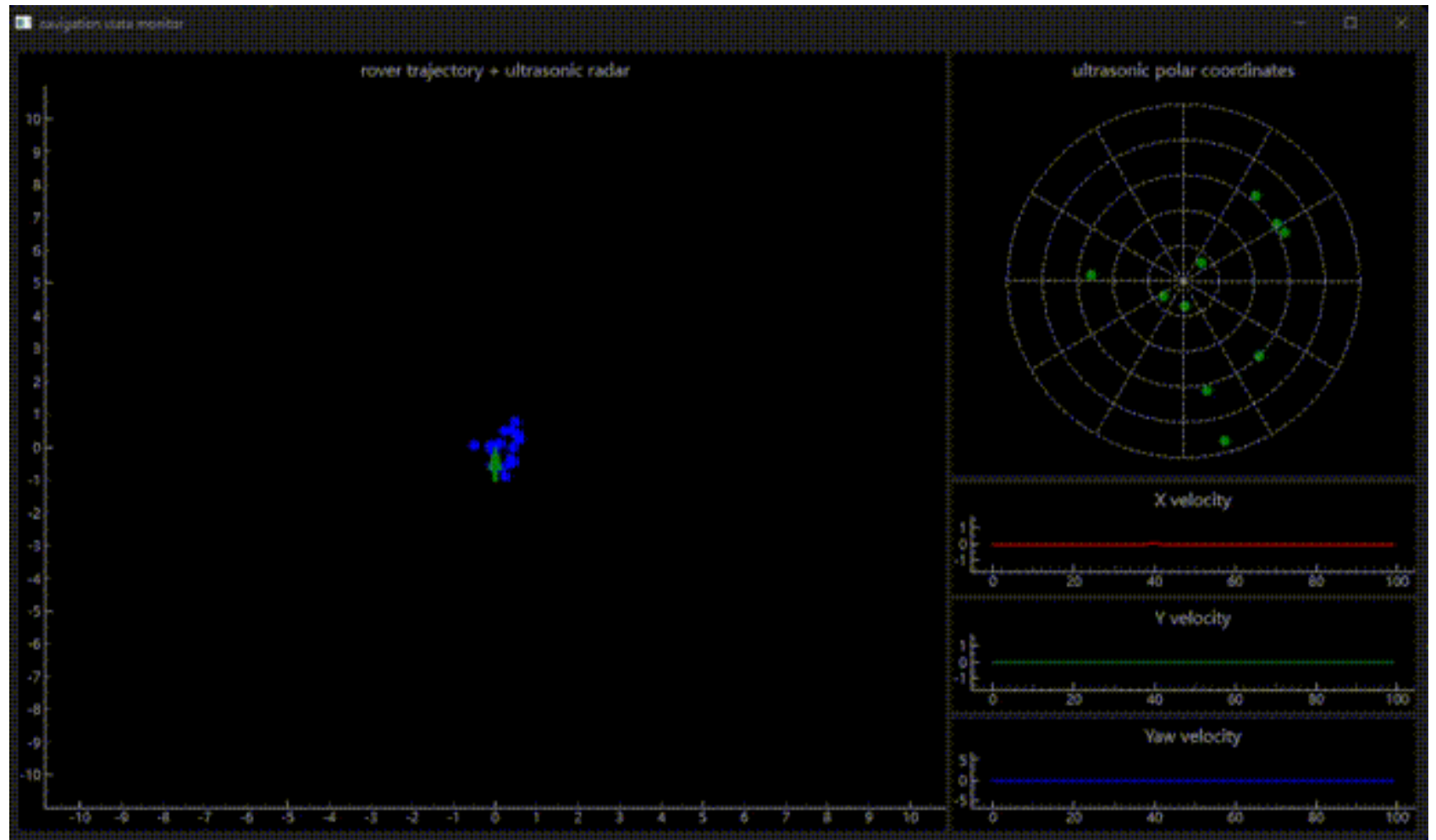
In the **parameter tuning** interface, users can input PID parameters in the interface and send them to the AGV, read parameters from the AGV, and load or save parameters from a local JSON file on the computer.



05 Navigation Visualization & Mission Planner

Navigation information visualization

We used QtGraph to plot the movement trajectory, speed curve and radar image of the AGV, enabling users to visually observe the navigation status of the AGV.

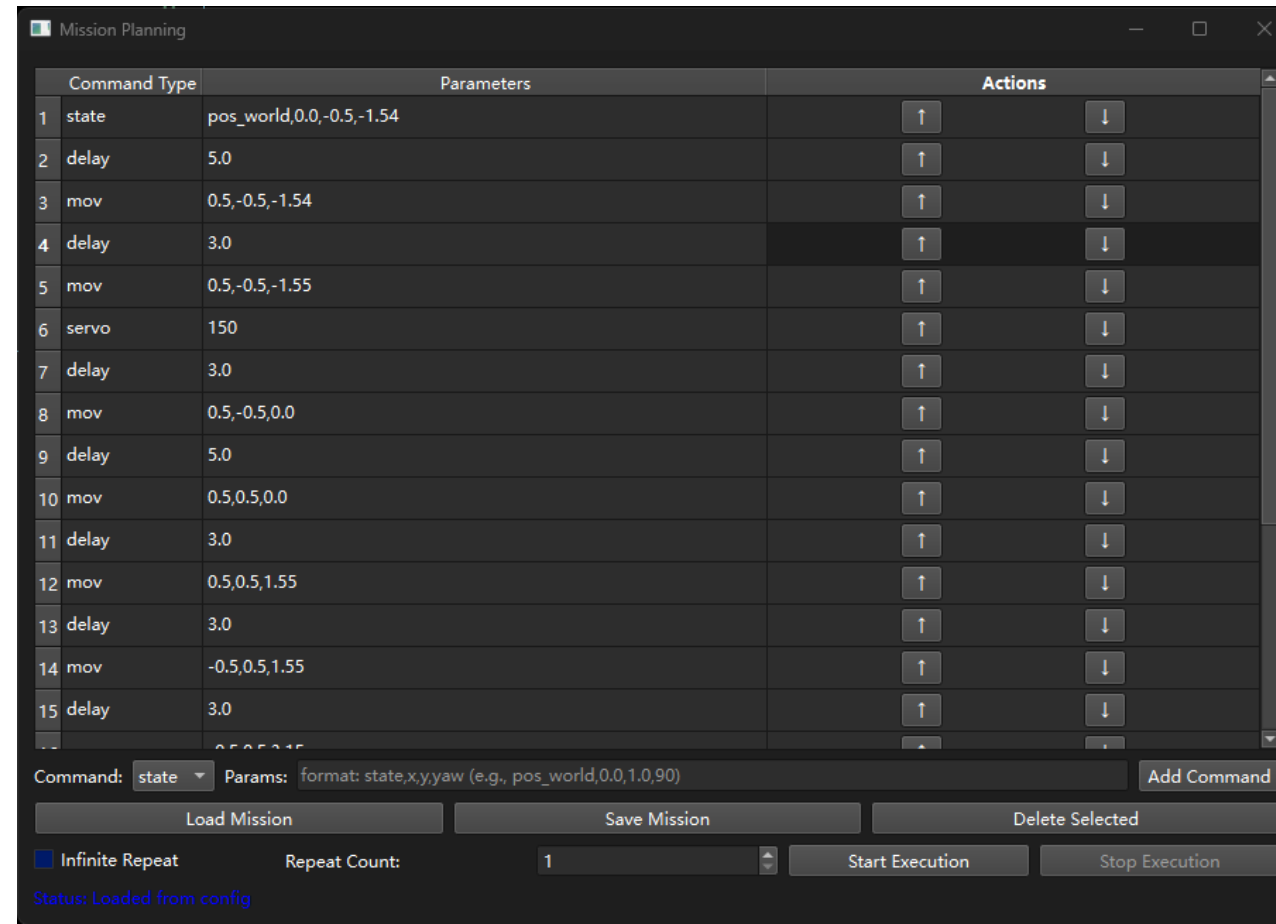


05 Navigation Visualization & Mission Planner

Mission planner

In order to ensure that the AGV can complete the automatic cruise operation, we have developed a mission planner for the AGV, which can control the vehicle to automatically complete the cruise task.

We have developed a simple and intuitive operation interface using QTableWidgetItem, which can read, edit and save the original json task code. Users do not need to understand the original code. They can complete the programming for the car task simply by following the instructions on the interface.



AGV vision process

In the software, we use OpenCV to read the transmitted images and allow users to activate the YOLO26 model for recognition, as well as utilize the function of using Aruco codes for navigation correction.



Part Six

Vision

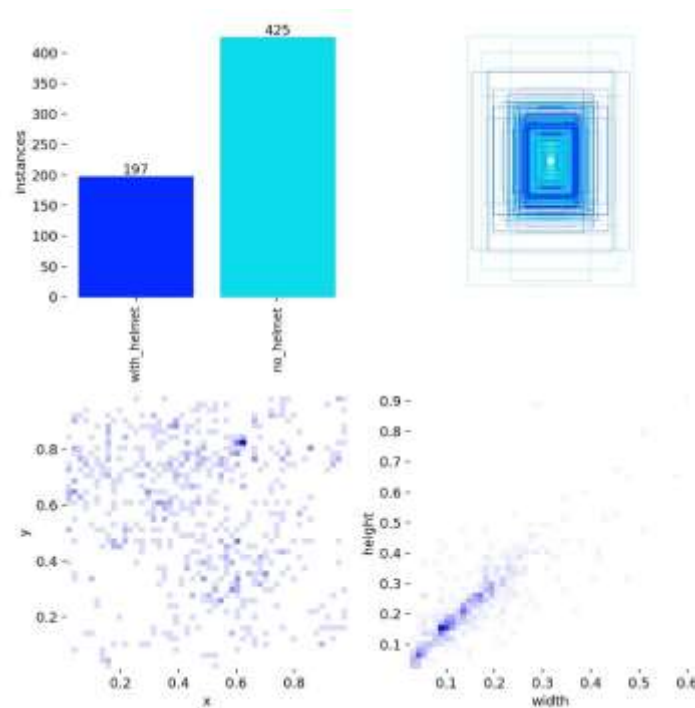
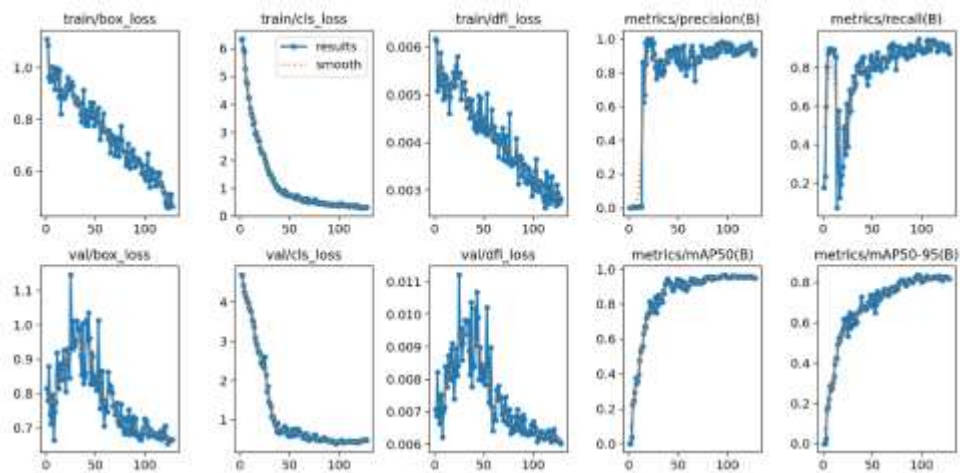
姚康晨

- YOLO Model: Dataset & Training Pipeline
- ArUco Code Positioning & Drift Correction

06 YOLO Model: Dataset & Training Pipeline

YOLO26 model training

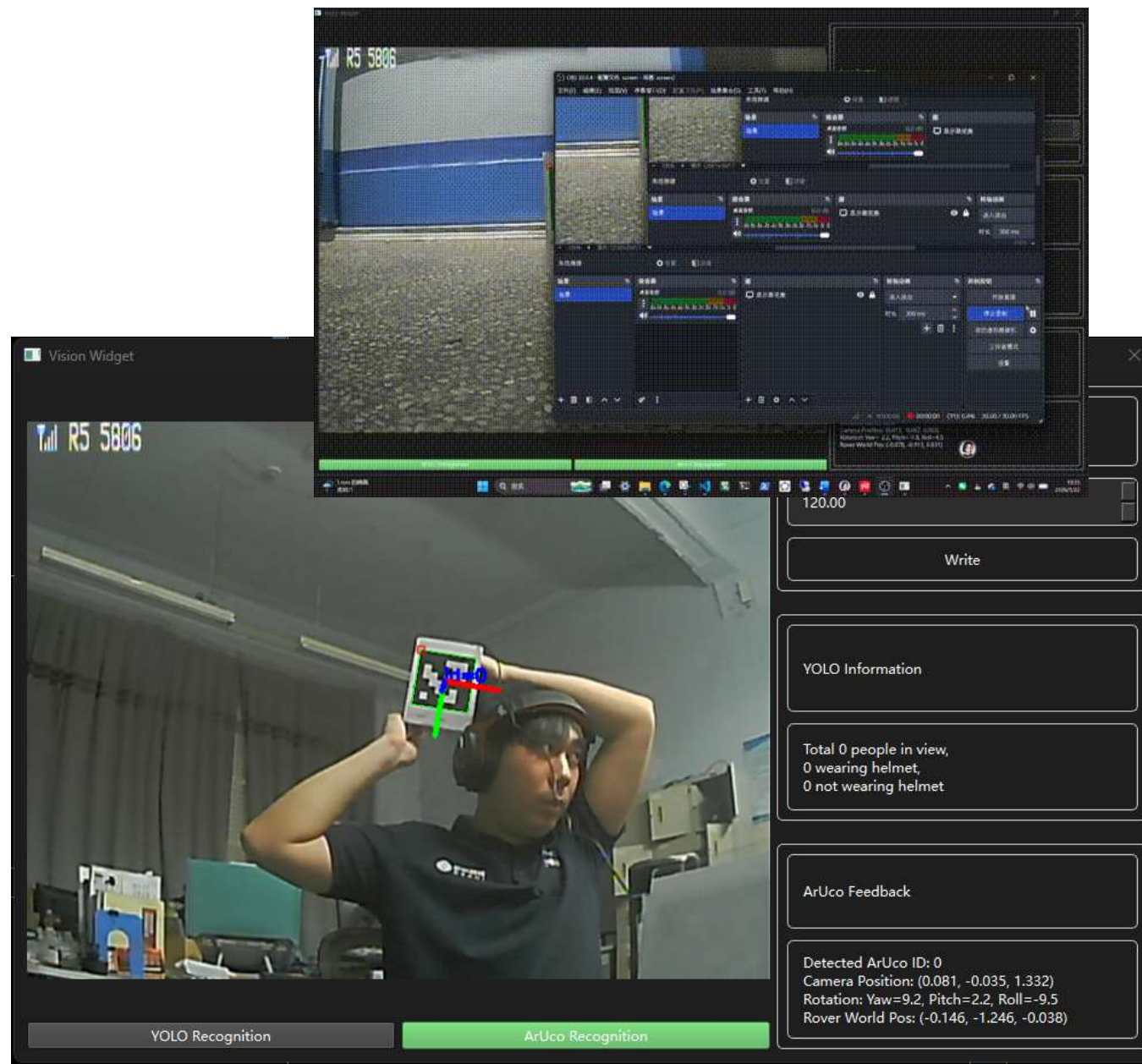
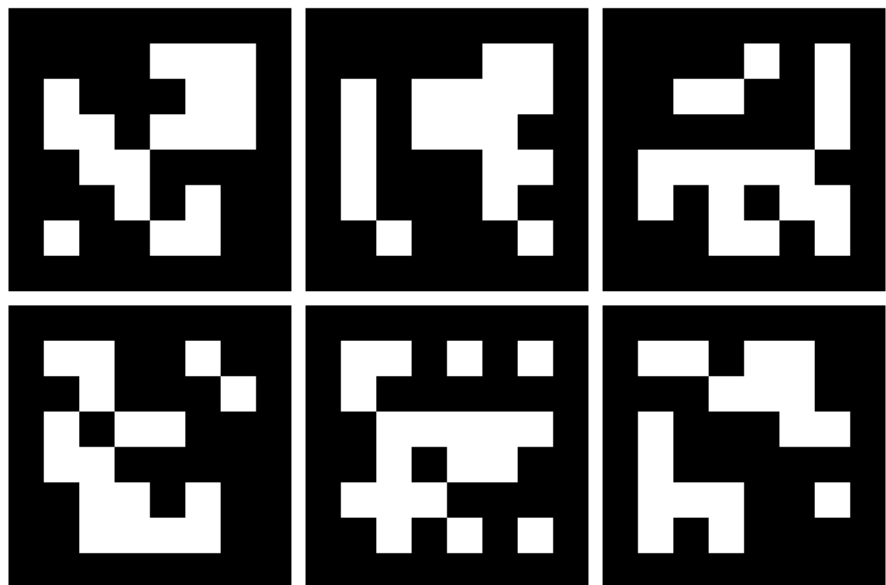
In order to complete the recognition of safety helmets, we collected and labeled the training set ourselves, and used the small-scale model of YOLO26 to complete the training on the server.



06 ArUco Code Positioning & Drift Correction

ArUco code positioning

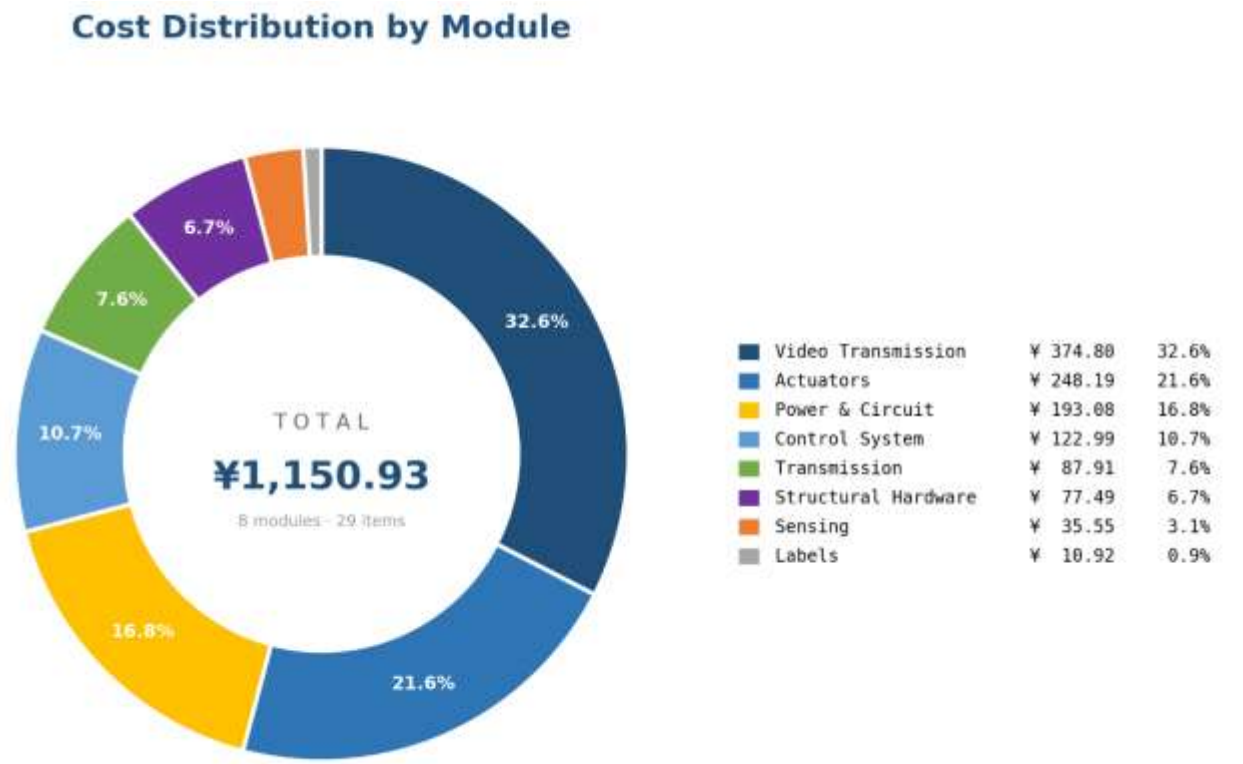
Because the built-in inertial navigation system of the car is prone to drift in terms of integration due to factors such as skidding, missing steps and oscillation, it is necessary to pre-arrange ArUco codes on the track and use its camera for calibration and positioning.



07 Budget

Project Expense Summary Report		
Mecanum-wheeled Autonomous Tracking Robot		
Team	Trace Rover	
Report Date	3 June 2026	
Currency	Chinese Yuan (CNY / ¥)	
Purpose	Reimbursement · Project Defense	
Breakdown by Module		
Module / Function	Item Count	Subtotal (CNY)
Video Transmission	3	374.80
Actuators	3	248.19
Power & Circuit	5	193.08
Control System	2	122.99
Transmission	5	87.91
Structural Hardware	7	77.49
Sensing	2	35.55
Labels	2	10.92
GRAND TOTAL (29 items)		¥1,150.93

Note: All amounts are invoice-verified (unit price × quantity + shipping). Based on 20 electronic invoices. See the Itemized List sheet for full details.



Mecanum-wheeled Autonomous Tracking Robot

Thank you for watching

汇报人 姚康晨

汇报日期 2026.5.23

Trace Rover Team